

System Design & Performance

Complete Interview Study Guide

Part I — Performance Fundamentals | Part II — Scaling & Distributed Architecture

How to use this guide

Each topic is explained from first principles, then followed by a short set of **likely interview questions with strong model answers** so you can rehearse out loud, not just re-read. A combined **Quick-Recall Cheat Sheet** for both parts sits at the very end — use it the night before, or the morning of, the interview.

Part I — Performance Fundamentals

1. Latency & Percentiles (P50, P90, P99)

What is latency?

Latency is the time a single request takes from the moment it's sent to the moment the response is received. It's made up of several stacked pieces:

- **Network transit** — time on the wire, both directions
- **Queueing time** — waiting for a free thread, connection, or CPU core
- **Processing time** — actual computation
- **I/O wait** — disk reads, database round-trips, calls to other services
- **Serialization** — encoding/decoding the payload (JSON, protobuf, etc.)

A request's total latency is the sum of these, and in a distributed system a single user-facing request often triggers a *chain* of internal calls, so latency compounds across hops.

Why averages are misleading

The average blends fast and slow requests into one number that can hide the actual user experience. Two failure modes:

1. **A skewed distribution.** Most systems have a long “tail” — the vast majority of requests are fast, but a small fraction take much longer (GC pauses, lock contention, a slow disk seek, a cold cache). The average gets pulled toward the fast majority and looks fine even though a meaningful number of real users are suffering.
2. **A bimodal distribution.** E.g., cache hits return in 2ms, cache misses in 200ms. The average (say, 20ms) doesn't describe *either* group of real users — nobody actually experiences “20ms.”

Core interview line: “Averages describe a number no user necessarily experiences. Percentiles describe what a given fraction of your actual users experience.”

Percentiles: P50, P90, P99

Sort all request latencies from fastest to slowest. The Pxx percentile is the value below which xx% of requests fall.

- **P50 (median):** half your requests are faster than this, half are slower — the “typical” experience.
- **P90:** 90% of requests are faster than this; the remaining 10% (the tail) are slower.
- **P99:** 99% of requests are faster than this — only 1% are slower, but that 1% is often where the real pain (and the real bugs) live.
- **P99.9 / P99.99:** used at very large scale, where even a “rare” 0.1% tail represents thousands or millions of real requests per day.

Why the tail matters more at scale

This is often called “**the tail at scale**” (from a well-known Google paper by Dean & Barroso):

- If you serve 10 million requests/day and 1% are slow (P99), that's **100,000 bad experiences per day** — not a rounding error.
- If a single page makes 20 backend calls, each with a 1% chance of hitting its own P99, the

probability that *at least one* is slow is roughly $1 - (0.99)^{20} \approx 18\%$. Tail latency **compounds** across fan-out calls — this is why large systems obsess over P99, not the average.

Likely interview questions

- **“Why not just use average latency for an SLA?”** → Averages hide tail behavior; at scale the tail represents a large absolute number of real users, and it often correlates with real problems (GC pauses, contention, slow queries) that you want surfaced, not averaged away.
- **“What’s the difference between P50, P90, and P99?”** → Explain via the sorted-distribution definition, anchored with a concrete example.
- **“How would you set a latency SLA?”** → As a percentile threshold (e.g. “P99 < 300ms”), not an average, because it makes an explicit promise about the worst-served fraction of users.

2. Throughput & Utilization

Throughput

Throughput is the number of operations completed per unit of time — requests/sec (QPS), transactions/sec (TPS), messages/sec. Where latency answers “how long does *one* thing take,” throughput answers “how many things can the system do *per second*.”

Little’s Law ties them together: $L = \lambda \times W$

- L = average number of requests in the system at once
- λ = arrival rate (throughput)
- W = average time each request spends in the system (latency)

Think of a system as a pipe: throughput is how much water flows through per second; latency is how long a drop takes to travel the pipe’s length.

Utilization

Utilization is the fraction of time a resource (CPU, disk, a thread pool, DB connections) is busy vs. idle: $\rho = \lambda / \mu$, where λ is arrival rate and μ is the max service rate.

Utilization & the latency curve

As utilization climbs toward 100%, queueing delay grows **asymptotically toward infinity**, not linearly. For a simple M/M/1 queue, wait time is proportional to $\rho / (1 - \rho)$:

Utilization	Relative wait
50%	x1
80%	x4
95%	x19
99%	x99

This produces the classic **“hockey stick” curve**: latency stays flat and low for a long stretch, then rockets upward near saturation.

Practical implication: never run production systems near 100% utilization on purpose. A common rule of thumb is to keep sustained utilization around 60–80%, leaving headroom to absorb spikes without falling off the cliff.

Likely interview questions

- **“If CPU utilization is at 95%, is that good?”** → No — it looks efficient but leaves no headroom; any burst pushes you further up the hockey-stick curve and latency blows up disproportionately.
 - **“How are latency and throughput related?”** → Little’s Law; pushing throughput higher increases queueing and therefore latency, especially near saturation.
-

3. Finding & Fixing Bottlenecks

Measure, don’t guess

Never optimize based on intuition alone — engineers are notoriously bad at guessing where time actually goes. Always profile first, then optimize the part the data points to. This connects to the **Theory of Constraints**: overall throughput is limited by the single slowest component; optimizing anything else does nothing for overall performance.

A practical workflow

1. **Define what “good” looks like** — SLIs/SLOs for latency percentiles, error rate, throughput.
2. **Look at dashboards** — CPU, memory, disk I/O, network, GC pauses, queue depths, DB query latency.
3. **USE method** (Utilization, Saturation, Errors) — sweep infra resources to spot what’s maxed out.
4. **RED method** (Rate, Errors, Duration) — a good default for request-driven service health.
5. **Load test** (k6, JMeter, Locust) to reproduce the problem and watch where latency bends upward.
6. **Profile** the specific slow component once you know where to look.

Profiling & distributed tracing

- **Profiling** (single process): samples a running program to show which functions consume the most CPU/memory — visualized as a **flame graph** (wider bars = more time).
- **Distributed tracing** (across services): a request produces a **trace** made of multiple **spans**, one per service hop. Tools like Jaeger, Zipkin, and OpenTelemetry stitch spans together so you can see exactly which hop is slow, instead of just “the checkout API is slow.”
- Look at the **critical path** — the longest chain of dependent spans — because that’s what determines total latency; parallel spans that finish early don’t matter.

Likely interview questions

- **“How do you find what’s slow in a system made of many microservices?”** → Distributed tracing to find the slow hop on the critical path, then profile that service.
 - **“You made a function 10x faster but overall latency didn’t change. Why?”** → It wasn’t the bottleneck; something else (often I/O-bound) was the actual constraint.
-

4. Database Performance

The N+1 Query Problem

Definition: fetch N parent records with one query, then — often due to ORM lazy loading — issue a *separate* query per parent for related data. Result: **1 + N** queries instead of 1–2.

```
posts = Post.objects.all()           # 1 query
for post in posts:
    print(post.author.name)          # +1 query per post, lazily triggered
```

Fixes: eager loading (`select_related` / `prefetch_related` in Django, `.includes()` in Rails, `JOIN FETCH` in JPA/Hibernate); batching with `WHERE id IN (...)`; the DataLoader pattern (common in GraphQL) to batch and cache lookups per request.

Database Indexes, Deep Dive

Purpose: avoid full table scans by letting the database jump directly to matching rows.

- **B-Tree** (default in most RDBMSs) — sorted, $O(\log n)$ lookups, supports range queries and sorting.
- **Hash index** — $O(1)$ exact-match only, no ranges or sorting.
- **Bitmap index** — efficient for low-cardinality columns, common in data warehouses.

Composite indexes & leftmost-prefix rule: an index on `(a, b)` serves queries filtering on `a` alone or `a AND b`, but **not** `b` alone.

Covering index: contains every column a query needs, so the DB can answer from the index alone (index-only scan).

Trade-off: indexes speed reads but slow writes (every INSERT/UPDATE/DELETE updates every index) and cost storage. Over-indexing is a real anti-pattern.

Reading the planner: `EXPLAIN ANALYZE` shows whether an index was used, estimated vs. actual row counts, and whether a sequential scan happened instead.

Likely interview questions

- **“When would an index exist but not get used?”** → Low selectivity, leading wildcard (`LIKE '%term'`), a function applied to the column without a matching functional index, type mismatches, or a table small enough that a full scan is cheaper.
- **“What’s the cost of adding an index?”** → Slower writes, more storage — always a trade-off.

Connection Pooling (Internal vs. External)

Opening a new DB connection per request is expensive (TCP handshake, auth, session setup) — often tens of milliseconds.

- **Internal pooling** — the pool lives inside the app process (HikariCP, SQLAlchemy pool). Simple, but each app instance has its own pool, so scaling instances horizontally can multiply total connections past the DB’s hard limit (Postgres defaults to ~ 100).
- **External pooling** — a proxy (**PgBouncer**, **ProxySQL**) sits between many app instances and the DB, multiplexing many client connections onto fewer real DB connections. Essential at scale and especially in serverless, where bursts of ephemeral instances would otherwise overwhelm the database.

Likely interview questions

- **“Why add PgBouncer if the app already pools connections?”** → Horizontal scaling multiplies independent internal pools; an external pooler caps the *total* connections the DB actually sees.

5. Caching Strategies & Patterns

Why cache

Caching trades a bit of data freshness for a large win in latency and throughput, by serving repeat reads from fast storage (memory) instead of a slower source.

Core patterns

Pattern	How it works	Trade-off
Cache-aside (lazy loading)	App checks cache first; on a miss, reads DB then writes into cache	Most common; risk of “thundering herd” on hot-key expiry
Read-through	Cache library handles the miss itself by loading from the DB	Cleaner app code, couples cache to data-loading logic
Write-through	Writes go to cache and DB together, synchronously	Always fresh, slower writes
Write-behind (write-back)	Writes hit cache immediately, flushed to DB asynchronously	Very fast writes, risk of loss if cache fails before flush
Write-around	Writes go straight to DB, cache fills only on reads	Good for write-heavy, rarely re-read data

Invalidation

“There are only two hard things in Computer Science: cache invalidation and naming things.”

- **TTL** — simplest; risk of staleness or too-aggressive expiry.
- **Explicit invalidation** — delete/update the key the moment underlying data changes.
- **Versioned keys** — bump a version on writes so old entries become unreachable and expire naturally.

Where to cache

Client-side (browser), CDN (edge caching), application-level in-memory (per-instance), distributed cache (Redis/Memcached, shared across instances). Each layer trades off latency, freshness, and how many requests it absorbs before hitting the origin.

Cache Hit Ratio

`Hit Ratio = Hits / (Hits + Misses)`

Driven by cache size vs. working-set size, TTL length, access-pattern skew (real traffic is often **Zipfian** — a small fraction of keys get most requests), and eviction policy (LRU, LFU, FIFO).

Likely interview questions

- **“What’s an acceptable hit ratio?”** → Context-dependent, but hot-path caches often target 90%+; a low ratio usually means an undersized cache, too-short TTL, or fragmented keys.
- **“How do you avoid a thundering herd on expiry?”** → Request coalescing, jittered TTLs, or serving stale data while refreshing in the background.

6. Vertical vs. Horizontal Scaling

Vertical scaling (“scaling up”)

Add more CPU/RAM/disk to a **single machine**.

Pros: simplest option, no distributed-systems complexity, no cross-node consistency issues, natural fit for inherently single-node systems (e.g., a relational DB’s primary writer). **Cons:** hard ceiling and disproportionate cost at the top end, single point of failure, often needs downtime to resize, diminishing returns.

Horizontal scaling (“scaling out”)

Add more machines and distribute load via a load balancer.

Pros: near-linear scalability in principle, better fault tolerance, commodity hardware, dynamic auto-scaling with traffic. **Cons:** real added complexity — load balancing, a strategy for state, data consistency across nodes, network overhead, and new distributed-system failure modes (partial failures, network partitions — the **CAP theorem** becomes relevant here).

Likely interview questions

- **“When would you choose vertical over horizontal?”** → Early-stage systems where simplicity matters most, workloads hard to parallelize, or when there’s still comfortable vertical headroom.
- **“How do you make horizontal scaling work for a formerly stateful service?”** → Externalize state (Redis-backed sessions instead of in-process memory) so any instance can serve any request — this is the exact bridge into Part II.

Part II — Scaling & Distributed Architecture

7. Statelessness: The Key to Horizontal Scaling

Stateful vs. stateless

A **stateful** service keeps request-specific data (session info, in-progress computation, uploaded files) in its own memory or local disk between requests. A **stateless** service keeps no such local memory — every request carries (or looks up) everything it needs, so any instance can handle any request equally well.

Why statelessness is the enabler

Horizontal scaling means “add more identical instances behind a load balancer.” That only works cleanly if **any** instance can serve **any** request. If user A’s session data only lives in the memory of server #3, then a load balancer *must* keep routing user A back to server #3 forever (a “sticky session”) — which defeats the purpose of having many interchangeable instances, creates hot spots, and means losing server #3 loses every session on it.

Statelessness removes that constraint entirely: instances become disposable, interchangeable, and trivially scalable up or down.

Where does the state actually go?

It doesn’t disappear — it moves to a shared, external location every instance can reach:

- **Distributed cache / session store** (Redis, Memcached) for session data
- **The database** for durable state
- **Object storage** (S3-style) for uploaded files, instead of local disk
- **The client itself** — e.g., a signed JWT that carries claims, so the server doesn’t need to look anything up at all

This is a core idea from the **12-Factor App** methodology: “*processes are stateless and share-nothing.*”

Sticky sessions — a stopgap, not a solution

Sticky sessions (routing a user to the same server based on a cookie or IP hash) can patch over a stateful design temporarily, but they reintroduce the problems statelessness is meant to solve: uneven load distribution, and lost data when that one instance restarts or fails.

Likely interview questions

- “**Why is a stateful service hard to scale horizontally?**” → Because a request can only be correctly served by the specific instance holding that user’s state, forcing sticky routing, which creates hot spots and a fragile single point of failure per user.
- “**How do you handle user sessions in a stateless architecture?**” → Externalize sessions to a shared store like Redis, or eliminate server-side session lookups entirely with a signed token (JWT) that the client presents on every request.

8. Load Balancers & Algorithms

What a load balancer does

Sits in front of a pool of backend instances and distributes incoming traffic across them, while continuously running **health checks** to detect and remove unhealthy instances from rotation automatically.

Layer 4 vs. Layer 7 load balancing

- **L4 (transport layer)** — routes based on IP address and TCP/UDP port only, without looking at the actual request content. Very fast, protocol-agnostic.
- **L7 (application layer)** — understands HTTP: can route based on URL path, headers, cookies, or hostname (e.g., `/api/*` to one service, `/static/*` to another). More flexible, slightly more overhead.

Load balancing algorithms

Algorithm	How it decides	Best for
Round robin	Cycles through servers in order	Simple, uniform workloads
Weighted round robin	Round robin, but bigger servers get proportionally more requests	Mixed-capacity fleets
Least connections	Sends to the server with fewest active connections	Long-lived / uneven-duration requests
Least response time	Sends to the server currently responding fastest	Latency-sensitive traffic
IP hash / consistent hashing	Hashes client IP (or key) to consistently pick the same server	Session affinity, cache locality
Random / weighted random	Picks randomly (optionally weighted by capacity)	Very large fleets where round robin overhead isn't worth it

Health checks

- **Active** — the load balancer periodically pings a health endpoint on each instance.
- **Passive** — the load balancer infers health from real traffic (e.g., a spike in errors or timeouts from one instance).

Real-world examples: **Nginx**, **HAProxy**, **Envoy**, and cloud-managed options like **AWS ALB** (L7) and **AWS NLB** (L4).

Likely interview questions

- **“Difference between L4 and L7 load balancing?”** → L4 only sees IP/port; L7 understands HTTP and can route on path/headers/cookies — useful for microservices routing, A/B testing, or blue-green deployments.
- **“When would you use least-connections over round robin?”** → When requests have very uneven durations — round robin can overload a server stuck on several slow requests, while least-connections naturally avoids that.
- **“How does a load balancer know a node is dead?”** → Failed active health checks and/or a pattern of failed real requests (passive detection); the node is pulled from rotation until it recovers.

9. Database Scaling (Read Replicas & Sharding)

Read replicas

A **primary (master)** database handles all writes; one or more **replicas (secondaries)** hold a continuously-updated copy of the data and serve read traffic. This scales *read* throughput horizontally without touching how writes work.

- **Async replication** — fast, but replicas can lag behind the primary by some delay.
- **Sync replication** — the primary waits for replica acknowledgment before confirming the write; safer, but slower.

Replication lag creates a real consistency problem: if a user writes data and immediately reads it back from a lagging replica, they may not see their own write (“read-your-own-writes” problem). Common fixes: route a user’s own reads to the primary right after they write, or route to a replica only after a lag threshold has passed.

Sharding (horizontal partitioning)

Read replicas don’t help write throughput or total storage — every replica still holds *all* the data.

Sharding splits the data itself across multiple independent database instances, each holding a different subset (“shard”), based on a **shard key**.

Common sharding strategies:

- **Range-based** — e.g., users A-M on shard 1, N-Z on shard 2. Simple, but can create hotspots if data isn’t evenly distributed.
- **Hash-based** — hash the shard key (e.g., `user_id`) to pick a shard. Spreads load evenly, but makes range queries across the whole dataset harder.
- **Directory-based** — a lookup service maps each key to its shard, giving flexibility to move data around at the cost of an extra lookup hop.

The hard parts of sharding:

- **Cross-shard joins/transactions** are difficult or impossible to do atomically — you often need to denormalize data or accept eventual consistency across shards.
- **Rebalancing** — adding a new shard means moving data around; naive `hash % N` schemes require reshuffling almost everything when `N` changes, which is why **consistent hashing** is commonly used instead (it minimizes the fraction of keys that need to move).
- **Hotspots** — a poorly chosen shard key (e.g., sharding by signup date when all traffic is on recent users) can leave one shard overloaded while others sit idle.

Replication vs. sharding — the key distinction

Replicas hold **full copies** of the same data (for read scaling and availability/failover). Shards hold **different subsets** of the data (for write scaling and storage scaling). Large systems typically use both together: shard the data, and give each shard its own read replicas.

Likely interview questions

- **“Difference between replication and sharding?”** → Replication copies all data to scale reads/availability; sharding splits data to scale writes/storage. They solve different problems and are often combined.
- **“How do you pick a shard key?”** → One with high cardinality and an even access distribution, ideally aligned with your most common query pattern, to avoid hotspots and minimize cross-shard queries.
- **“How do you handle ‘read your own write’ with replicas?”** → Route the user’s immediate post-write reads to the primary, or track a “read-your-writes” token/timestamp and wait for a replica to catch up to it.

10. Content Delivery Networks (CDNs)

What a CDN does

A CDN is a geographically distributed network of caching servers (**edge locations** or **Points of Presence**) that sit physically closer to end users than your origin server does. A request is routed (via DNS or anycast) to the nearest edge node; if that node has the content cached (a **cache hit**), it serves it directly — otherwise (a **cache miss**), it fetches from the origin, caches it, and serves it, so the *next* nearby user gets a hit.

Why it helps

- **Latency** — physical distance to the server is a hard floor on latency (light itself takes time); serving from a nearby edge node instead of a distant origin cuts this dramatically.
- **Origin load** — the origin server only has to handle cache misses, not every single request, which also protects it during traffic spikes.

What gets cached

Classically: static assets — images, CSS, JS bundles, video segments. Increasingly: API responses too, when they're cacheable (with proper `Cache-Control` / `ETag` headers) and don't need to be personalized per user.

CDN cache invalidation

Same fundamental problem as any cache: TTLs for automatic expiry, plus an explicit **purge/invalidate API** call for when content changes before the TTL is up (e.g., after deploying new static assets, or updating a product image).

Likely interview questions

- **"How does a CDN reduce latency?"** → By serving content from a location physically close to the user instead of a single distant origin, cutting network round-trip time.
- **"How do you invalidate a CDN cache when content changes?"** → Short TTLs plus an explicit purge call on deploy, or cache-busting via versioned file names/URLs (e.g., `app.a1b2c3.js`) so you never need to invalidate old ones — the URL itself changes.

11. Edge Computing

Beyond caching: compute at the edge

A CDN edge node traditionally just serves cached bytes. **Edge computing** goes a step further: it runs actual application logic on those same edge nodes, physically close to the user, instead of routing every request all the way back to a centralized origin server.

Examples of platforms: **Cloudflare Workers**, **Lambda@Edge** / **CloudFront Functions**, **Fastly Compute**.

What kinds of logic belong at the edge

Lightweight, low-latency, often per-request logic that doesn't need the full backend or database:

- Authentication/authorization checks before a request even reaches the origin
- A/B testing and feature-flag routing

- Personalization or geolocation-based content variants
- Image resizing/optimization on the fly
- Simple request/response rewriting, redirects, or even full server-side rendering of a page's shell

The trade-off

Edge functions typically run in restricted, lightweight runtimes (fast cold starts, limited execution time, no persistent local state), so they're excellent for small, stateless, fast operations — but not a replacement for your core backend's heavier business logic or database-bound work.

Likely interview questions

- **“What's the difference between a CDN and edge computing?”** → A CDN serves cached static content from a nearby location; edge computing runs actual code at that same nearby location, enabling dynamic, per-request logic without a round trip to a central origin.
- **“What kind of logic is a good fit for the edge?”** → Small, fast, mostly stateless operations — auth checks, redirects, personalization, A/B testing — not heavy database-bound business logic.

12. Asynchronous Processing & Background Jobs

Why go async

Not every piece of work needs to finish before the client gets a response. Sending a confirmation email, generating a large report, resizing an uploaded image, or processing a payment webhook can all happen *after* the user-facing request returns — improving perceived latency and freeing up request-handling capacity.

The core building blocks

- **Message queue** (RabbitMQ, AWS SQS, Kafka) — the producer (web server) drops a job description onto the queue and immediately returns a response to the user; it doesn't wait for the job to run.
- **Background worker** (Celery, Sidekiq, Bull, or a Kafka consumer group) — a separate process that pulls jobs off the queue and executes them independently, at its own pace.

This **decouples** the producer from the consumer: the web server doesn't need to know or care how long the job takes, and the queue absorbs sudden bursts of work so the workers can process them at a steady, sustainable rate instead of getting overwhelmed instantly.

Common patterns

- **Fire-and-forget** — enqueue and move on; no result is ever sent back.
- **Polling** — the client periodically checks a status endpoint (“is my report ready yet?”).
- **Webhook callback** — the system calls the client back (or another service) once the job finishes.
- **Pub/sub fan-out** — one event triggers multiple independent consumers (e.g., “order placed” triggers billing, inventory, and notification services simultaneously).

Trade-offs to name explicitly

- **Eventual consistency** — the work isn't done the instant the request returns; downstream state catches up shortly after.
- **Idempotency is essential** — most queues guarantee “at-least-once” delivery, meaning a job

might run more than once (e.g., after a worker crash mid-processing and a retry); jobs must be written so running them twice doesn't double-charge a card or send two emails.

- **Dead-letter queues (DLQs)** — a job that repeatedly fails gets moved to a separate queue for inspection instead of endlessly retrying or silently vanishing.
- **Monitoring** — queue depth and processing latency need their own dashboards; a growing backlog is an early warning sign that workers can't keep up with producers.

Likely interview questions

- **"When would you make something asynchronous instead of synchronous?"** → When the work doesn't need to complete before responding to the user, is slow/unpredictable in duration, or benefits from being retried independently of the original request.
- **"How do you guarantee a job isn't processed twice?"** → You generally can't guarantee exactly-once delivery cheaply, so instead you design the job to be **idempotent** — e.g., using a unique idempotency key to detect and skip a duplicate execution.
- **"What's a dead-letter queue for?"** → Catching jobs that fail repeatedly, so they don't block the queue or get silently dropped, and can be inspected/replayed manually.

13. Microservices vs. Monolith

Monolith

A single deployable application containing all modules/business logic in one codebase and process, typically backed by one database.

Pros: simple to develop, test, deploy, and debug early on; easy transactions (one database, real ACID guarantees); no network calls between internal modules, so no distributed-systems failure modes to worry about internally. **Cons:** hard to scale individual parts independently (you scale the whole thing even if only one feature is hot); a large codebase gets harder to build/deploy/reason about as the team grows; a bug or resource leak in one module can take down the entire application; the whole app is locked into one technology stack.

Microservices

The application is split into independently deployable services, each typically owning its own data store, communicating over the network (REST, gRPC, or async messaging).

Pros: independent scaling and independent deployment per service; teams can own services end-to-end (aligns with **Conway's Law** — system structure tends to mirror team structure); fault isolation (one service crashing doesn't necessarily take down the others); freedom to pick different tech stacks per service where it makes sense. **Cons:** real operational complexity — service discovery, deployment pipelines, and monitoring multiply; network latency and partial-failure modes now exist *between* what used to be simple in-process function calls; no more easy cross-module transactions — consistency across services usually requires eventual consistency and patterns like the **saga pattern** instead of a single ACID transaction; debugging requires distributed tracing instead of just reading a stack trace.

Choosing between them

A common and well-respected piece of advice is to **start with a monolith** and split out services only once a specific, real pain point justifies it (an independent scaling need, a team-ownership boundary, a reliability isolation requirement) — not preemptively. Splitting too early, without addressing the underlying data-ownership and communication challenges, produces a **"distributed monolith"**: all the network complexity and failure modes of microservices, with

none of the independent-deployability benefit, because services still end up tightly coupled and must be deployed together.

Likely interview questions

- **“When do you split a monolith into microservices?”** → When a specific part of the system needs to scale independently, be owned by a separate team, use a different tech stack, or be isolated for reliability — not simply “because microservices are considered best practice.”
 - **“What’s a distributed monolith, and why is it bad?”** → Services that are physically separate (deployed independently) but logically still tightly coupled (shared database, synchronous chains of calls, can’t be deployed independently in practice) — you pay all of the network/complexity cost of microservices with none of the benefit.
 - **“How do you handle a transaction spanning two microservices?”** → The **saga pattern** — a sequence of local transactions, each with a defined compensating action to undo it if a later step fails — since a classic two-phase-commit distributed transaction is usually too slow/fragile across service boundaries.
-

14. Serverless Computing

What “serverless” actually means

You write and deploy individual functions; the cloud provider handles provisioning, scaling (including scaling all the way down to zero when idle), patching, and infrastructure management entirely. This model is often called **FaaS (Functions as a Service)** — AWS Lambda, Google Cloud Functions, and Azure Functions are the major examples.

Defining characteristics

- **Event-driven invocation** — a function runs in response to a trigger: an HTTP request, a file upload, a queue message, a scheduled timer.
- **Automatic, elastic scaling** — the platform spins up as many concurrent instances of your function as needed, and back down to zero when there’s no traffic.
- **Pay-per-execution billing** — you’re billed for actual invocations and execution time, not for idle capacity sitting around waiting for traffic.

Pros

No server management or capacity planning; automatic scaling handled for you; genuinely cost-efficient for spiky or intermittent workloads, since you pay nothing while idle; fast to ship small, focused pieces of logic independently.

Cons

- **Cold starts** — when a function hasn’t run recently, spinning up a fresh instance adds noticeable latency to that particular request; this is the most commonly asked serverless interview topic.
- **Execution time limits** — most platforms cap how long a single invocation can run, making serverless a poor fit for long-running processes.
- **Statelessness by design** — no persistent local state between invocations, which pushes you right back to the same statelessness/externalized-state principles from Section 7.
- **Database connection pressure** — this directly reconnects to Part I’s connection-pooling topic: a burst of concurrent function invocations can each try to open its own database connection, quickly exceeding the database’s connection limit; the standard fix is an external connection pooler (e.g., PgBouncer) or a serverless-native database proxy sitting in front of the database.
- **Vendor lock-in** and **harder local testing/debugging** compared to a normal running server.

- **Cost can flip the other way** for steady, high-throughput workloads — a constantly-busy reserved server can end up cheaper than paying per-invocation at high volume.

When to use it (and when not to)

Good fit: sporadic or bursty workloads, event-driven glue code, webhook handlers, scheduled/cron-style jobs, on-upload image processing. Poor fit: latency-sensitive services under constant high-throughput steady-state traffic, or long-running/stateful processes.

Likely interview questions

- **“What’s a cold start, and how do you mitigate it?”** → The added latency when a new function instance has to initialize before handling its first request; mitigations include “provisioned concurrency” (keeping a warm pool of pre-initialized instances) or periodic “keep-warm” pings, at the cost of losing some of the pure pay-per-use savings.
- **“When would serverless *not* be a good fit?”** → Constant, high-volume, latency-sensitive traffic (where reserved/always-on compute is both cheaper and avoids cold starts), or long-running/stateful processes that exceed platform time limits.
- **“How does serverless affect database connection management?”** → Each concurrent invocation can behave like a brand-new client wanting its own DB connection, so bursts can exhaust the database’s connection limit fast — solved with an external pooler or proxy in front of the database, not by pooling inside each ephemeral function.

15. Key Takeaways & Mental Models

Pulling the whole picture together, a few mental models recur across every topic in both parts of this guide:

1. **Measure, don’t guess.** Every optimization decision — from finding a bottleneck to deciding whether to shard a database — should be driven by data (profiling, tracing, metrics), not intuition.
2. **Percentiles over averages.** Tail latency is where real user pain and real bugs live; design and alert on P99, not the mean.
3. **Statelessness is the enabling constraint for horizontal scaling.** Every scaling technique in Part II — load balancing, sharding, serverless — assumes or works toward instances/functions that don’t hold irreplaceable local state.
4. **Push work to where it’s cheapest.** Cache what you can (closer to the user is better: CDN/edge beats origin), and defer what doesn’t need to happen synchronously (async processing/queues) so the critical path stays short.
5. **Every architecture choice is a trade-off, not a best practice.** Consistency vs. availability (CAP), latency vs. complexity (microservices), cost vs. control (serverless vs. reserved compute) — a good interview answer names the trade-off explicitly rather than declaring a universal winner.
6. **Start simple, scale deliberately.** A monolith on a single, vertically-scaled machine is a legitimate, often *correct* starting architecture. Complexity (sharding, microservices, multi-region edge logic) should be introduced in response to a real, specific bottleneck — not preemptively.

Combined Quick-Recall Cheat Sheet

Part I — Performance Fundamentals

Concept	One-line answer
Why not average latency?	Hides the tail; the tail = a large absolute number of bad experiences at scale
p99	99% of requests are faster than this value; the remaining 1% is the tail
Utilization & latency	Latency grows non-linearly (hockey-stick) as utilization approaches 100%
Bottleneck hunting	Measure first (tracing/profiling), never guess; fix only the actual constraint
N+1 problem	1 query for parents + N queries for children when 1-2 would do; fix with eager loading/joins
Index trade-off	Faster reads, slower writes, more storage; leftmost-prefix rule for composite indexes
Connection pooling	Reuse DB connections; external poolers (PgBouncer) cap total connections across many app instances
Cache-aside	Check cache → miss → read DB → populate cache; the most common pattern
Cache hit ratio	hits / (hits + misses); driven by cache size, TTL, and access skew
Vertical scaling	Bigger machine; simple but has a ceiling and is a single point of failure
Horizontal scaling	More machines; scales further but adds distributed-systems complexity

Part II — Scaling & Distributed Architecture

Concept	One-line answer
Statelessness	No local server memory between requests — any instance can serve any request
Sticky sessions	A stopgap for stateful design, not a real solution — reintroduces hot spots
L4 vs L7 load balancing	L4 = IP/port only; L7 = understands HTTP, can route on path/headers/cookies
Least connections	Routes to the server with fewest active connections — good for uneven request durations
Read replicas	Copies of <i>all</i> data to scale reads/availability; watch for replication lag
Sharding	Splits <i>different subsets</i> of data across nodes to scale writes/storage; hard part is the shard key and rebalancing
CDN	Caches content at edge locations near users to cut latency and origin load
Edge computing	Runs actual code (not just cached bytes) close to the user for fast, lightweight logic
Async processing	Queue + worker decouples slow work from the request path; requires idempotency
Dead-letter queue	Catches jobs that keep failing instead of retrying forever or vanishing silently
Monolith vs. microservices	Start monolith for simplicity; split only for a specific scaling/ownership/isolation need
Distributed monolith	Services deployed separately but still tightly coupled — worst of both worlds
Serverless	Pay-per-execution, auto-scales to zero; watch for cold starts and DB connection bursts
Saga pattern	Chain of local transactions with compensating actions, used instead of cross-service 2PC

Guide compiled from the two-part video series you shared, covering both sessions' full topic and timestamp lists. Good luck with the interview.